DOCKER

DOCKER INTEGRA TODO LO NECESARIO PARA EMPAQUETAR NUESTRO SOFTWARE, CREANDO UNA IMAGEN

UNA VEZ CREADA LA IMAGEN, SE PUEDE EJECUTAR UN CONTENEDOR

EL CONTENEDOR SE PUEDE EJECUTAR ALLÁ DONDE HALLA UNA MÁQUINA CON DOCKER

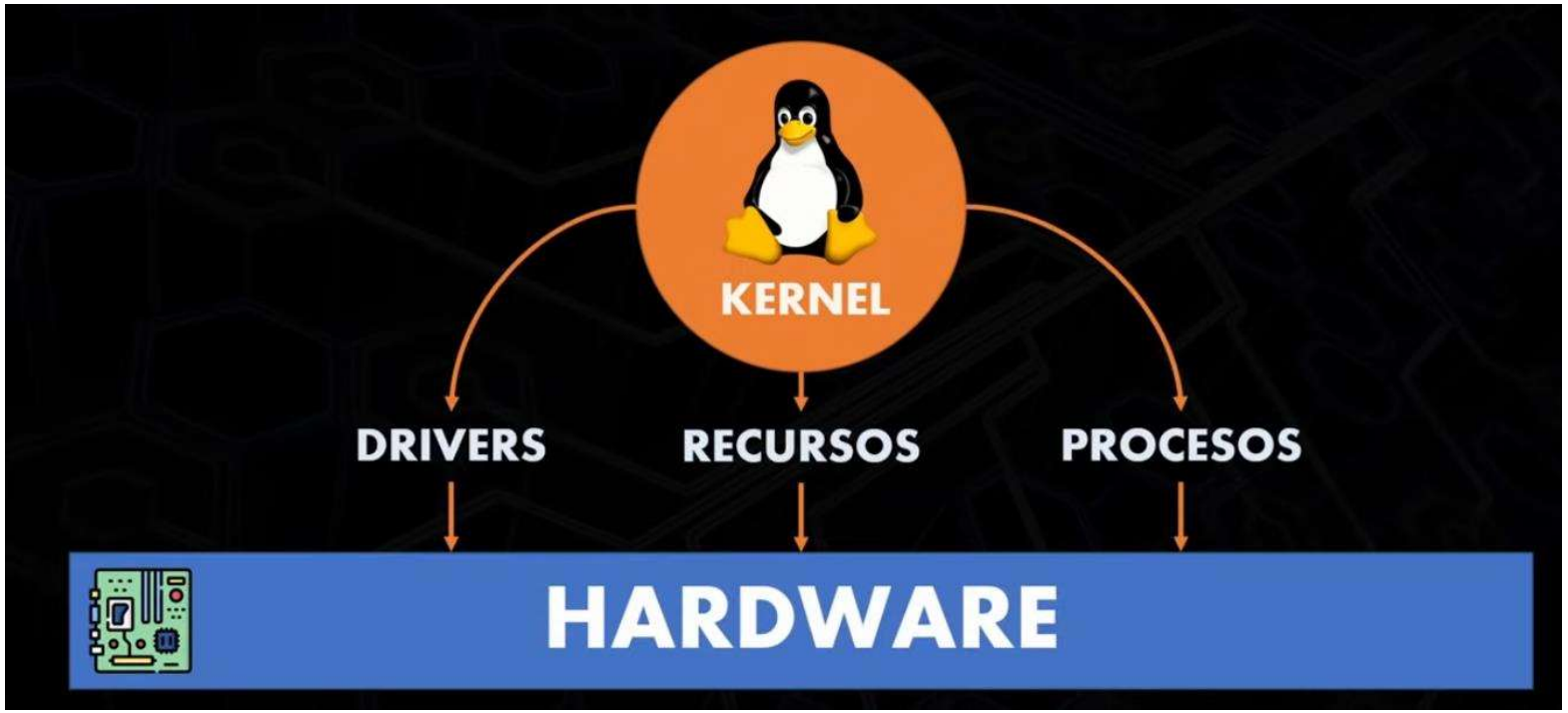
DOCKER VS MÁQUINAS VIRTUALES

EN LAS MÁQUINAS VIRTUALES, LA CONFIGURACIÓN REQUIERE DE UN SISTEMA OPERATIVO COMPLETO

LA MÁQUINA VIRTUAL VIRTUALIZA EL HARDWARE, MIENTRAS QUE LOS CONTENEDORES VIRTUALIZAN A TRAVÉS DEL SISTEMA OPERATIVO Y SON MÁS LIGEROS

EL KERNEL ES COMÚN PARA DISTINTOS CONTENEDORES. AHÍ RESIDE LA CLAVE

DOCKER VS MÁQUINAS VIRTUALES



DOCKER VS MÁQUINAS VIRTUALES



DOCKER VS MÁQUINAS VIRTUALES



DOCKER VS MÁQUINAS VIRTUALES

LOS GRUPOS DE CONTROL Y LOS ESPACIOS DE NOMBRE SON LAS CARACTERÍSTICAS DEL KERNEL DE LINUX QUE NOS DAN EL AISLAMIENTO Y LA ASGINACIÓN DE RECURSOS

DE MANERA QUE A TRAVÉS DE FUNCIONES INTERNAS DEL SISTEMA OPERATIVO, SE GESTIONAN RECURSOS HARDWARE EXCLUSIVOS PARA CADA CONTENEDOR, SIN NECESIDAD DE TENER EL SO COMPLETO

DOCKERFILE

DOCKERFILE ES EL FICHERO QUE DESCRIBE CÓMO COMPONER LA IMAGEN Y LA PLANTILLA DEL PROYECTO

PARTIENDO DE UNA IMAGEN BASE (YA SEA UN SO O UN ENTORNO COMO JDK) SE AGREGA EL PROYECTO COMPILADO (JAR)

LA IMAGEN QUEDA LISTA PARA USARSE EN NUESTRO ENTORNO

DOCKERFILE

EJEMPLO DE IMAGEN PARTIENDO DE LA ÚLTIMA VERSIÓN DE JAVA, AGREGAMOS NUESTRA APP Y ESPECIFICAMOS EL PUNTO DE ENTRADA

```
FROM OPENJDK

ADD REST.JAR RESTCONT.JAR

ENTRYPOINT JAVA -JAR RESTCONT.JAR
```

DOCKER COMANDOS

DOCKER IMAGES => LISTADO DE IMÁGENES LOCAL

DOCKER IMAGES RM <IMAGEN>:<VERSIÓN> => LISTADO DE IMÁGENES LOCAL

DOCKER PS => CONTENEDORES EN EJECUCIÓN

DOCKER BUILD -T <IMAGEN> . => CONSTRUIMOS UNA IMAGEN

DOCKER RUN -P 8085:9456 <IMAGEN>

DOCKER PULL <NOMBRE_IMAGEN> => DESCARGA LA ÚLTIMA VERSIÓN

DOCKER TAG <IMAGEN> USUARIO/REPO:VERSION => CREAMOS UNA VERSIÓN

DOCKER PUSH USUARIO/REPO:VERSION => SUBIMOS VERSIÓN A DOCKERHUB